PROJET IA & COGNITION · 5DS DATA SCIENCE · ESPRIM

Voyara — Guide de réalisation du moteur IA

14 étapes + 2-bis, chacune avec son prompt prêt pour Claude Code

Version 2 — transport intégré : vols, voitures de location, transferts

De la demande en langage naturel à l'itinéraire personnalisé, cohérent et adaptable — du vol aller au vol retour : « Votre voyage. À votre façon. »

- **Module 1** · Compréhension du besoin (+ ville de départ, besoins transport) → étape 4
- **Module 2** · Recherche & collecte (+ recherche structurée transport) → étapes 1, 2, 2-bis, 5
- **Module 3** · Recommandation Top-5 (+ Top-3 transport) → étape 6
- **Module 4** · Construction de l'itinéraire (arrivée, départ, transferts) → étape 7
- **Module 5** · Budget & optimisation (vols, voiture, transferts) → étape 7
- **Module 6** · Agent intelligent (outils transport) → étape 8

Équipe TravelMinds · Boutheina Rjab · Ali Ben Jannet · Ons Alouini · Ghayth Mahmoud
Choix retenus : LLM hybride (Claude API + Ollama local gratuit) · catalogue Voyara (activités, hébergements, vols, voitures, transferts) + OpenStreetMap + Open-Meteo

Ce qui change dans la version 2

La version 1 ne branchait le moteur IA que sur les **activités**, les **hébergements**, les **POI** et la **météo**. Les vols, voitures et transferts du catalogue étaient ignorés et les déplacements seulement estimés au kilomètre. La version 2 les intègre **de bout en bout**, sans casser ce qui est déjà fait.

> **Décision clé :** les offres de transport ne passent **pas** par l'index vectoriel. Prix, places et horaires changent en permanence et se cherchent par critères exacts (origine, destination, date, passagers) : le service `ai` interroge `catalog` **en direct** via `/internal/*`. Seuls les hébergements rejoignent la recherche sémantique.

Étape	Statut v2	Changement
0, 1, 2	Inchangées	Déjà réalisées. Complétées par l'étape 2-bis (aucune à refaire).
2-bis	**Nouvelle**	Endpoints internes catalog : export hébergements, recherche vols / voitures / transferts ; aéroports localisés ; mise à jour de l'architecture et de toute la documentation.
3	Mineur	Rappel : `LLM_PROVIDER=ollama` permet de tout faire gratuitement.
4	Modifiée	DemandeVoyage : ville_depart, besoins_transport, preferences_transport ; 25 demandes transport dans le jeu d'évaluation.
5	Modifiée	Index : + hébergements via l'export 2-bis. Nouveau client de recherche structurée transport.
6	Modifiée	Top-3 explicable par type de transport demandé.
7	Modifiée	Jours d'arrivée/départ, transferts, choix voiture ou transferts dans CP-SAT, nouveaux postes de budget.
8	Modifiée	Outils rechercher_transport, proposer_reservation étendu ; 4e scénario « transport ».
9 à 13	Modifiées	Interface (étapes vol/transfert, budget), back-office (paramètres transport), évaluation, sécurité (aucun passeport/permis demandé), démo.

> **Règle inchangée :** l'agent **propose** des vols, voitures et transferts, avec un lien vers la page du site ; il ne réserve ni ne paie jamais lui-même.

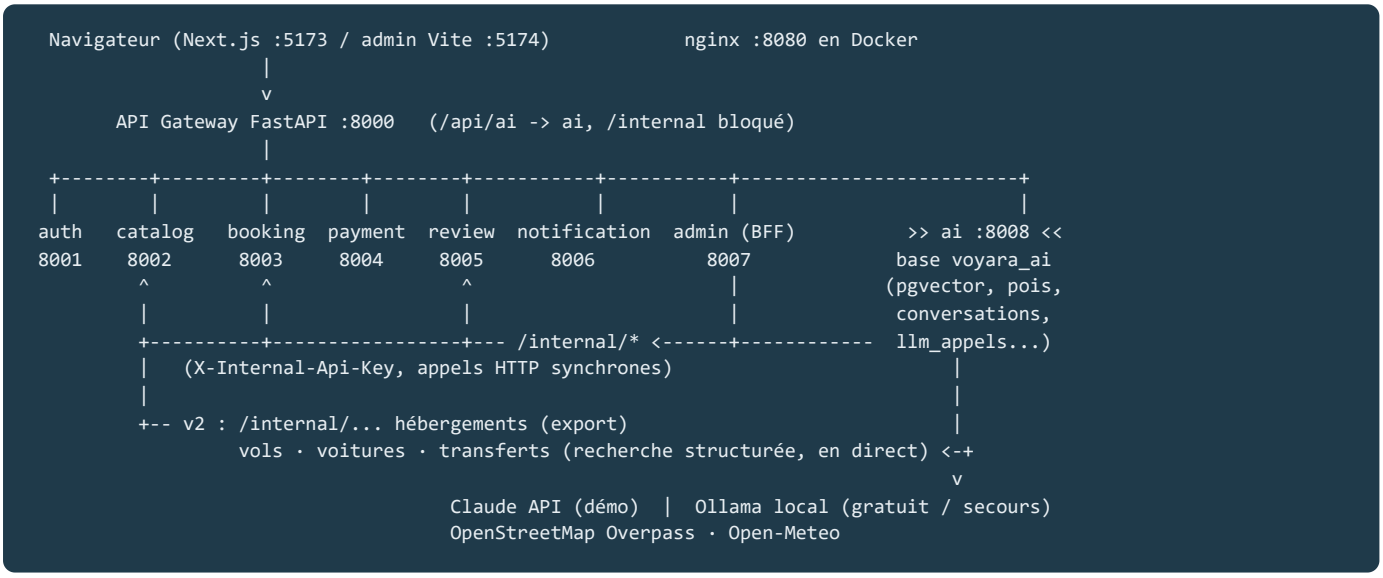
1. Point de départ et architecture cible

Ce qui existe déjà : une plateforme de réservation complète (7 microservices FastAPI, gateway, Next.js + admin Vite, Stripe, i18n fr/en/ar). Le catalogue contient activités (prix, durée, localisation, latitude/longitude, catégorie), créneaux, hébergements, vols, voitures et transferts.

Ce qui manquait : aucune brique IA ; pas de restaurants, plages, monuments ni météo ; pas d'attributs comme « effort de marche » ou « public famille » ; aucun accès IA aux offres de transport.

> **Principe directeur :** le LLM *comprend* et *explique* ; les *faits* (prix, horaires, places, y compris des vols) viennent de la base, le *classement* d'un score mesurable et le *planning* d'un solveur d'optimisation. C'est ce qui rend l'itinéraire fiable, explicable et évaluable — et c'est la réponse d'un data scientist à la question « comment éviter les hallucinations ? ».

Architecture cible (v2)



Pourquoi un nouveau microservice ai ?

- **Isolation :** le moteur peut être lent ou en panne sans bloquer la réservation et le paiement.
- **Database-per-service respecté :** vecteurs, POI, conversations et journaux LLM vivent dans `voyara_ai` ; les données métier (activités, hébergements, vols, voitures, transferts) sont lues via `/internal/*`, jamais par jointure SQL.
- **pgvector plutôt qu'une base vectorielle externe :** une seule technologie de base, suffisante pour quelques milliers de documents.
- **Transport en recherche structurée :** pas d'embeddings pour des offres dont le prix et les places changent ; on interroge la source de vérité au moment de la demande.
- **LLM hybride :** Claude pour la qualité en démo, Ollama local pour le coût nul, le hors-ligne et la comparaison scientifique.

Les étapes (v2)

#	Étape	Module couvert	Durée
0	Cadrage, conventions et architecture cible	Transverse	½ journée
1	Données : points d'intérêt et météo	Module 2 — Recherche & collecte	2 jours
2	Enrichissement sémantique du catalogue	Modules 2 & 3	1–2 jours
2-bis	**Hébergements et offres de transport pour l'IA**	Module 2 + architecture	1 jour
3	Service ai et couche LLM hybride	Socle technique du moteur	2 jours
4	Module 1 — Comprendre le voyageur (NLU)	Module 1 — Compréhension du besoin	3 jours
5	Module 2 — Recherche sémantique, RAG et recherche transport	Module 2 — Recherche & collecte	3–4 jours
6	Module 3 — Recommandation Top-5 et Top-3 transport	Module 3 — Recommandation personnalisée	3 jours
7	Modules 4 & 5 — Itinéraire et optimisation sous contraintes	Module 4 — Itinéraire · Module 5 — Budget	4–5 jours
8	Module 6 — Agent intelligent	Module 6 — Adaptation intelligente	3–4 jours
9	Interface voyageur : « Planifier mon voyage »	Interface Voyageur	3 jours
10	Back-office : paramétrage et analytics IA	Back-office VOYARA	2 jours
11	Évaluation globale, tests et CI	Démarche Data Science	2 jours
12	Sécurité, coûts et RGPD du moteur IA	Transverse	1–2 jours
13	Démo, rapport et soutenance	Livraison	2 jours

2. Comment utiliser ce guide avec Claude Code

Le cycle d'une étape

1. Ouvre Claude Code dans le dossier `Voyara`, sur une branche dédiée (ex. `ia/etape-04`).
2. Active le **plan mode**, colle le prompt de l'étape tel quel.
3. Lis le plan proposé ; corrige-le si besoin ; valide.
4. Laisse Claude Code coder, lancer les tests et écrire le rapport.
5. Lis le rapport dans `docs/ia/rapports/`, vérifie toi-même la section 7 « Ce que je dois vérifier ».
6. Commit, puis ouvre une **nouvelle session** pour l'étape suivante (contexte propre ; les rapports font le lien).

Structure commune de chaque prompt

- **En-tête de contexte** (gris) : fichiers à lire, plan mode, contraintes permanentes du projet.
- **Corps** (noir) : ce qu'il faut construire, précisément.
- **Rapport obligatoire** (terracotta) : ce que Claude Code a fait, pourquoi, quelles alternatives il a écartées, comment il a vérifié.

Modèle de rapport (inscrit dans CLAUDE.md à l'étape 0)

Section	Contenu attendu
1. Objectif	Ce que l'étape devait accomplir.
2. Réalisé	Chaque fichier créé/modifié et son rôle.
3. Choix techniques	Option retenue, alternatives, raison de l'écart — pour chaque décision.
4. Vérification	Commandes PowerShell lancées et sortie réelle.
5. Métriques	Chiffres obtenus et jeu de données utilisé.
6. Limites	Dette technique, risques, cas non gérés.
7. À vérifier par moi	Pas à pas manuel pour l'étudiant.
8. Suite	Étape suivante recommandée.

> **Règles d'or :** une étape à la fois ; ne jamais accepter « les tests passent » sans la sortie de commande ; aucune clé API dans le code ou les rapports (fichier `.env` uniquement, saisi par toi) ; les rapports d'étape sont la matière première du rapport de PFE.

> **Sans budget API :** tout le guide fonctionne avec **Ollama en local** (`LLM_PROVIDER=ollama`). Les métriques du rapport sont alors celles d'Ollama ; la comparaison Claude vs Ollama devient une perspective.

Ordre et priorités

MVP de la présentation (must) : étapes 0, 1, 2, **2-bis**, 3, 4, 5, 6, 7, 8, 9 — saisie de la demande, recherche, recommandations, transport, itinéraire, budget et modification. **Should :** 10, 11, 12. **Indispensable avant le jury :** 13. Si le temps manque, garde l'étape 11 au moins en version réduite : sans chiffres, pas de démarche data science.

○ Cadrage, conventions et architecture cible

Transverse | Durée indicative : ½ journée

Objectif

Poser les règles du jeu avant d'écrire la moindre ligne d'IA : où vit le moteur, comment Claude Code doit travailler, et comment chaque étape est tracée.

Pourquoi ce choix (regard d'expert)

En data science, ce qui n'est pas tracé n'existe pas devant un jury. Un fichier CLAUDE.md garantit que chaque session de Claude Code respecte les mêmes règles (rapport obligatoire, français, pas de jointure entre services). Le nouveau service **ai** isole le moteur IA : il peut tomber sans casser la réservation.

Livrables

- CLAUDE.md à la racine (règles + convention de rapport)
- docs/ia/README.md, docs/ia/journal.md, docs/ia/rapports/
- ADR « Service ai + base voyara_ai + pgvector + LLM hybride » dans docs/architecture.md
- Phase « Moteur IA » (étapes 1 à 13) ajoutée à docs/plan.md

> **v2 :** étape déjà réalisée. Sa mise à jour (périmètre transport, ADR, plan) est faite par l'étape 2-bis.

> **Critère de « fini » :** CLAUDE.md existe, l'ADR est rédigé, docs/plan.md liste les étapes avec effort/priorité.

Prompt à coller dans Claude Code — étape 0

Contexte : projet Voyara (dépôt actuel). Avant tout, lis CLAUDE.md, docs/architecture.md, docs/plan.md, docs/mcd.md, docs/ia/ et TOUS les rapports existants dans docs/ia/rapports/. Travaille en plan mode : propose d'abord ton plan (fichiers touchés, choix, risques) et attends ma validation avant d'écrire du code.
Contraintes permanentes : code, commentaires et messages en français ; champs métier en français (nom, prix, ...) ; découpage par domaine, aucune jointure SQL entre services (appels HTTP /internal/* avec INTERNAL_API_KEY via ServiceClient) ; réutiliser libs/common/voyara_common (config, auth, internal, database, pagination, exceptions, logging, limiter) ; textes d'interface via i18n fr/en/ar ; l'agent IA propose mais ne réserve ni ne paie jamais ; je suis sous Windows/PowerShell.

Étape 0 — Cadrage du moteur IA de Voyara.
Lis aussi AgentIA_Voyara/PresProjCom.pdf : c'est la présentation du moteur IA (6 modules : compréhension, recherche & collecte, recommandation Top-5, itinéraire, budget & optimisation, agent).

À faire :
1. Crée CLAUDE.md à la racine : résumé de l'architecture (7 services + gateway + front), conventions (français, database-per-service, /internal/* + INTERNAL_API_KEY, i18n fr/en/ar, voyara_common), commandes utiles (scripts/dev.ps1, tests), et la règle : « chaque tâche se termine par un rapport dans docs/ia/rapports/ selon le modèle de docs/ia/README.md ».
2. Crée docs/ia/README.md (vision du moteur IA, schéma des 6 modules, modèle de rapport), docs/ia/journal.md et le dossier docs/ia/rapports/.
3. Ajoute un ADR dans docs/architecture.md : nouveau microservice `ai` (port 8008, base `voyara_ai`, extension pgvector), appels synchrones vers catalog/review/booking via /internal/*, route gateway /api/ai, couche LLM hybride (Claude API par défaut, Ollama local en secours). Contexte, décision, alternatives (IA dans catalog, base vectorielle externe type Qdrant, LLM unique), conséquences.
4. Ajoute à docs/plan.md une phase « Moteur IA » avec les étapes 1 à 13 de ce guide (effort S/M/L, priorité must/should/nice).
N'écris aucun code applicatif à cette étape.

RAPPORT OBLIGATOIRE — à la fin, crée docs/ia/rapports/etape-00-cadrage.md avec :
  1. Objectif de l'étape.
  2. Ce que tu as fait : chaque fichier créé/modifié, avec son rôle en une ligne.
  3. Choix techniques : pour CHAQUE décision, l'option retenue, les alternatives envisagées et pourquoi tu les as écartées.
  4. Vérification : commandes PowerShell exécutées et sortie réelle (extraits). N'écris jamais qu'un test passe si tu ne l'as pas lancé ; si quelque chose échoue, dis-le.
  5. Métriques obtenues (si applicable), avec le jeu de données utilisé.
  6. Limites, dette technique, risques.
  7. Ce que je dois vérifier moi-même (pas à pas).
  8. Étape suivante recommandée.
Mets aussi à jour docs/ia/journal.md (une ligne : date, étape, résumé) et coche la tâche dans docs/plan.md.
Termine en affichant le résumé du rapport dans le chat et propose un message de commit.

1 Données : points d'intérêt et météo

Module 2 — Recherche & collecte | Durée indicative : 2 jours

Objectif

Le catalogue ne contient pas de restaurants, plages, musées, monuments, souks, ni de météo. Un itinéraire en a besoin.

Pourquoi ce choix (regard d'expert)

La qualité d'un système de recommandation est bornée par ses données. On collecte depuis OpenStreetMap (API Overpass, licence ODbL, légale et gratuite) plutôt que de scraper des sites commerciaux : c'est reproductible et défendable. La météo vient d'Open-Meteo (sans clé). Le notebook d'EDA montre au jury que tu connais tes données (couverture, valeurs manquantes, biais).

Livrables

- Table pois dans voyara_ai (type, nom, nom_en, nom_ar, ville, latitude, longitude, horaires, prix_moyen, tags, source, maj_le)
- services/ai/scripts/collecter_pois.py (Overpass, par ville, idempotent)
- Client Open-Meteo avec cache en base (prévisions par ville et date)
- notebooks/01_eda_donnees.ipynb + figures exportées pour le rapport

v2 : étape déjà réalisée. Le type « aeroport » (utile aux transferts) est ajouté par l'étape 2-bis si nécessaire.

Critère de « fini » : ≥ 300 POI répartis sur au moins 5 villes, météo disponible pour Djerba, notebook exécuté sans erreur.

Prompt à coller dans Claude Code — étape 1

Contexte : projet Voyara (dépôt actuel). Avant tout, lis CLAUDE.md, docs/architecture.md, docs/plan.md, docs/mcd.md, docs/ia/ et TOUS les rapports existants dans docs/ia/rapports/. Travaille en plan mode : propose d'abord ton plan (fichiers touchés, choix, risques) et attends ma validation avant d'écrire du code.
Contraintes permanentes : code, commentaires et messages en français ; champs métier en français (nom, prix, ...) ; découpage par domaine, aucune jointure SQL entre services (appels HTTP /internal/* avec INTERNAL_API_KEY via ServiceClient) ; réutiliser libs/common/voyara_common (config, auth, internal, database, pagination, exceptions, logging, limiter) ; textes d'interface via i18n fr/en/ar ; l'agent IA propose mais ne réserve ni ne paie jamais ; je suis sous Windows/PowerShell.

Étape 1 — Collecte des données touristiques (POI + météo).
Le service `ai` n'existe pas encore : crée seulement le minimum dans services/ai/ (models.py, database.py, config.py calqués sur services/catalog/app) pour porter la table, le scaffold complet viendra à l'étape 3. Ajoute la base `voyara_ai` à infra/init.sql et à scripts/create_databases.py.

À faire :
1. Modèle SQLAlchemy `Poi` : type (restaurant | plage | musee | monument | souk | parc | cafe), nom, nom_en, nom_ar, ville, latitude, longitude, horaires_ouverture (texte OSM brut + version normalisée JSON), prix_moyen (nullable), tags (JSON), source, source_id (unique), maj_le.
2. Script services/ai/scripts/collecter_pois.py : requêtes Overpass par ville (Djerba/Houmt Souk, Tunis, Sidi Bou Saïd, Sousse, Hammamet, Monastir, Tozeur, Kairouan), mapping des tags OSM vers nos types, upsert idempotent sur source_id, respect du rate limit Overpass (pause + retry), option --ville.
3. Client météo services/ai/app/meteo.py (Open-Meteo, sans clé) avec cache en table `meteo_cache` (ville, date, temp_min, temp_max, precipitations, code_meteo, maj_le) et expiration 6 h.
4. Notebook notebooks/01_eda_donnees.ipynb : volumes par ville/type, taux de valeurs manquantes (horaires, nom_ar, prix), carte des points, doublons ; exporte les figures dans docs/ia/figures/.
5. Tests pytest du mapping OSM → Poi et du cache météo (httpx mocké).
Explique dans le rapport pourquoi OSM plutôt que du scraping de TripAdvisor/Google (licence, CGU, stabilité).

RAPPORT OBLIGATOIRE — à la fin, crée docs/ia/rapports/etape-01-donnees-poi-meteo.md avec :
  1. Objectif de l'étape.
  2. Ce que tu as fait : chaque fichier créé/modifié, avec son rôle en une ligne.
  3. Choix techniques : pour CHAQUE décision, l'option retenue, les alternatives envisagées et pourquoi tu les as écartées.
  4. Vérification : commandes PowerShell exécutées et sortie réelle (extraits). N'écris jamais qu'un test passe si tu ne l'as pas lancé ; si quelque chose échoue, dis-le.
  5. Métriques obtenues (si applicable), avec le jeu de données utilisé.
  6. Limites, dette technique, risques.
  7. Ce que je dois vérifier moi-même (pas à pas).
  8. Étape suivante recommandée.
Mets aussi à jour docs/ia/journal.md (une ligne : date, étape, résumé) et coche la tâche dans docs/plan.md.
Termine en affichant le résumé du rapport dans le chat et propose un message de commit.

2 Enrichissement sémantique du catalogue

Modules 2 & 3 | Durée indicative : 1–2 jours

Objectif

Donner à chaque activité des attributs exploitables par l'IA : public visé, effort de marche, intérieur/extérieur, moment idéal de la journée, thèmes.

Pourquoi ce choix (regard d'expert)

Le scénario « je préfère moins marcher » de la présentation est impossible sans un attribut *niveau_marche*. On fait un pré-étiquetage (LLM, ou règles hors-ligne si aucune clé n'est disponible) puis une validation humaine dans l'admin (fiable) : c'est la démarche classique « weak labelling + human in the loop ». Le catalogue reste propriétaire de ses données : les tags vivent dans catalog, pas dans ai.

Livrables

- Colonnes/table de tags dans catalog (thèmes, public, niveau_marche 1–3, interieur, moment_ideal, duree déjà existante)
- Endpoint /internal/activities/export (paginé, champs complets : description, lat/lng, catégorie, tags)
- Script de pré-étiquetage produisant un CSV à valider
- Écran admin de validation des tags (fr/en/ar)

v2 : inchangée. Les tags restent réservés aux activités ; le transport n'a pas besoin d'étiquettes sémantiques.

Critère de « fini » : 100 % des activités actives ont des tags validés ; export interne testé.

Prompt à coller dans Claude Code — étape 2

Contexte : projet Voyara (dépôt actuel). Avant tout, lis CLAUDE.md, docs/architecture.md, docs/plan.md, docs/mcd.md, docs/ia/ et TOUS les rapports existants dans docs/ia/rapports/. Travaille en plan mode : propose d'abord ton plan (fichiers touchés, choix, risques) et attends ma validation avant d'écrire du code. Contraintes permanentes : code, commentaires et messages en français ; champs métier en français (nom, prix, ...) ; découpage par domaine, aucune jointure SQL entre services (appels HTTP /internal/* avec INTERNAL_API_KEY via ServiceClient) ; réutiliser libs/common/voyara_common (config, auth, internal, database, pagination, exceptions, logging, limiter) ; textes d'interface via i18n fr/en/ar ; l'agent IA propose mais ne réserve ni ne paie jamais ; je suis sous Windows/PowerShell.

Étape 2 — Enrichissement du catalogue pour l'IA.
Le service catalog (services/catalog/app/models.py) a Activity avec titre, description, prix, duree, localisation, latitude, longitude, category_id. L'endpoint actuel /internal/activities/batch ne renvoie que id/titre/prix/photos : il ne suffit pas.

À faire :
1. Dans catalog, ajoute les attributs IA d'une activité : themes (liste parmi plage, culture, histoire, nature, gastronomie, aventure, detente, shopping, nautique, desert), public (famille, couple, solo, groupe, enfants), niveau_marche (1 faible, 2 moyen, 3 élevé), interieur (bool), moment_ideal (matin, apres_midi, soir, indifferent), tags_valides (bool). Choisis colonnes JSON ou table d'association et justifie-le. Migration propre (catalog n'utilise pas Alembic : explique comment tu migres sans perdre les données).
2. Endpoint /internal/activities/export paginé (protégé par require_internal_key) renvoyant toutes les infos utiles à l'IA, + paramètre modifie_depuis pour la synchronisation incrémentale.
3. Script services/catalog/scripts/pre_etiqueter.py : envoie titre+description à un LLM (pour l'instant appel direct simple, la couche hybride arrive à l'étape 3) ou, sans clé, applique des règles hors-ligne ; écrit un CSV de propositions, sans rien écrire en base.
4. Route admin + écran dans frontend/apps/admin : liste des activités, tags proposés, édition, bouton « Valider » ; textes via @voyara/i18n (fr/en/ar).
5. Tests : export interne (clé absente → 401/403), validation des valeurs autorisées.

RAPPORT OBLIGATOIRE — à la fin, crée docs/ia/rapports/etape-02-enrichissement-catalogue.md avec :
  1. Objectif de l'étape.
  2. Ce que tu as fait : chaque fichier créé/modifié, avec son rôle en une ligne.
  3. Choix techniques : pour CHAQUE décision, l'option retenue, les alternatives envisagées et pourquoi tu les as écartées.
  4. Vérification : commandes PowerShell exécutées et sortie réelle (extraits). N'écris jamais qu'un test passe si tu ne l'as pas lancé ; si quelque chose échoue, dis-le.
  5. Métriques obtenues (si applicable), avec le jeu de données utilisé.
  6. Limites, dette technique, risques.
  7. Ce que je dois vérifier moi-même (pas à pas).
  8. Étape suivante recommandée.
Mets aussi à jour docs/ia/journal.md (une ligne : date, étape, résumé) et coche la tâche dans docs/plan.md.
Termine en affichant le résumé du rapport dans le chat et propose un message de commit.

2-bis Hébergements et offres de transport pour l'IA

NOUVEAU v2 | Module 2 + architecture | Durée indicative : 1 jour

Objectif

Rendre lisibles par le service ai les hébergements, vols, voitures de location et transferts du catalogue, et mettre l'architecture et toute la documentation à jour pour ce nouveau périmètre.

Pourquoi ce choix (regard d'expert)

Un itinéraire « réalisable » commence à l'arrivée du vol et finit au départ. Mais les offres de transport sont volatiles (places, prix) et se cherchent par critères exacts : les indexer en vecteurs produirait des résultats périmés et non vérifiables. On expose donc des endpoints internes de **recherche structurée** que le service ai appelle en direct — la source de vérité reste catalog. Les hébergements, eux, sont descriptifs et stables : ils rejoignent l'index sémantique de l'étape 5.

Livrables

- catalog : export interne des hébergements (paginé, modifie_depuis)
- catalog : recherche interne vols, voitures, transferts (filtres, disponibilité, prix total)
- POI de type « aeroport » si les aéroports ne sont pas encore localisés
- Docs à jour : avenant ADR + schéma dans docs/architecture.md, CLAUDE.md, docs/ia/README.md, docs/plan.md, docs/mcd.md si besoin

> **Critère de « fini » :** les 4 endpoints internes répondent et sont testés (401/403, filtres, pagination) ; les tests de l'étape 2 restent verts ; l'architecture documentée correspond au code.

Prompt à coller dans Claude Code — étape 2-bis

```
Contexte : projet Voyara (dépôt actuel). Avant tout, lis CLAUDE.md, docs/architecture.md, docs/plan.md, docs/mcd.md, docs/ia/ et TOUS les rapports existants dans docs/ia/rapports/. Travaille en plan mode : propose d'abord ton plan (fichiers touchés, choix, risques) et attends ma validation avant d'écrire du code.
Contraintes permanentes : code, commentaires et messages en français ; champs métier en français (nom, prix, ...) ; découpage par domaine, aucune jointure SQL entre services (appels HTTP /internal/* avec INTERNAL_API_KEY via ServiceClient) ; réutiliser libs/common/voyara_common (config, auth, internal, database, pagination, exceptions, logging, limiter) ; textes d'interface via i18n fr/en/ar ; l'agent IA propose mais ne réserve ni ne paie jamais ; je suis sous Windows/PowerShell.

Étape 2-bis — Ouvrir les hébergements, vols, voitures de location et transferts au moteur IA, et mettre à jour l'architecture.
Lis aussi AgentIA_Voyara/Guide_Moteur_IA_Voyara.pdf (version 2) : le périmètre du moteur IA s'étend aux offres de transport. Les étapes 0, 1 et 2 sont déjà faites : ne les refais pas, complète-les.

Principe : les offres de transport ne sont PAS indexées en vecteurs. Prix, places et horaires changent souvent et se cherchent par critères exacts (origine, destination, date, nombre de passagers) : le service ai interrogera catalog en direct. Seuls les hébergements rejoignent l'index sémantique de l'étape 5.

À faire :
1. Inventaire : lis services/catalog/app/models.py, ses routes et docs/mcd.md ; liste dans ton plan les modèles réels des hébergements, vols, voitures et transferts (noms, champs, disponibilités, lien avec une ville ou un aéroport). N'invente aucun champ : si une information manque (coordonnées d'un aéroport, capacité d'un véhicule, ville d'un transfert...), propose l'ajout minimal et justifie-le. Migration sans perte de données, comme à l'étape 2.
2. Endpoints internes dans catalog, protégés par require_internal_key, en suivant le nommage existant des routes /internal/* :
   - hébergements : export paginé avec modifie_depuis (comme /internal/activities/export) et les champs utiles à l'IA (nom, description, ville, latitude, longitude, prix_nuit, capacité, équipements, note).
   - vols : recherche par origine, destination, date (tolérance ± N jours), nb_passagers → vols avec places restantes, horaires de départ et d'arrivée, escales, prix par passager et prix total.
   - voitures : recherche par lieu de prise et de retour (ville ou aéroport), dates, nb_places, boîte/catégorie optionnelles → véhicules disponibles, prix par jour et prix total.
   - transferts : recherche par lieu de départ, lieu d'arrivée (aéroport ↔ ville/hôtel), date-heure, nb_passagers, bagages → offres avec prix et durée.
   Réponses paginées (voyara_common.pagination), prix en DT, dates ISO 8601, aucun champ personnel.
3. Si les aéroports (et ports/gares utiles) ne sont pas déjà localisés — nécessaire pour les transferts et l'heure d'arrivée — ajoute le type « aeroport » au collecteur de l'étape 1 (services/ai/scripts/collecter_pois.py) et relance-le pour les villes concernées, sans dupliquer les POI existants.
4. Tests pytest dans catalog : clé absente → 401, clé invalide → 403 (cohérent avec la correction 422 → 401 de libs/common), filtres (dates, places, capacité), pagination, aucun vol complet ni véhicule indisponible renvoyé.
5. Documentation à mettre à jour (obligatoire) :
   - docs/architecture.md : avenant à l'ADR du service ai (« le moteur IA couvre activités, hébergements, POI, météo ET vols, voitures, transferts ; le transport passe par une recherche structurée en direct via catalog, sans index vectoriel ; l'agent ne réserve jamais »), schéma d'architecture mis à jour (flux ai → catalog /internal/* pour le transport), alternatives écartées (indexer le transport dans pgvector, appeler directement des API de compagnies externes) et pourquoi.
   - CLAUDE.md : périmètre du moteur IA et liste des nouveaux endpoints internes.
   - docs/ia/README.md : schéma des 6 modules avec la place du transport dans chacun (compréhension : ville de départ et besoins transport ; recherche : recherche structurée ; recommandation : Top-3 transport ; itinéraire : arrivée/départ et transferts ; budget : nouveaux postes ; agent : nouveaux outils).
   - docs/plan.md : ajoute la tâche 2-bis et mets à jour la description des étapes 4 à 13 selon le guide v2.
   - docs/mcd.md : seulement si tu as ajouté des colonnes ou des tables.
   - Ne réécris pas les rapports déjà livrés (étapes 0, 1, 2) : ajoute à la fin de chacun une courte section « Avenant v2 » seulement si un point y est devenu faux.
6. Vérifie que l'export d'activités et le pré-étiquetage de l'étape 2 fonctionnent toujours (tests existants verts, sortie réelle dans le rapport).

RAPPORT OBLIGATOIRE — à la fin, crée docs/ia/rapports/etape-02bis-transport-catalogue.md avec :
  1. Objectif de l'étape.
  2. Ce que tu as fait : chaque fichier créé/modifié, avec son rôle en une ligne.
  3. Choix techniques : pour CHAQUE décision, l'option retenue, les alternatives envisagées et pourquoi tu les as
```

```
écartées.
  4. Vérification : commandes PowerShell exécutées et sortie réelle (extraits). N'écris jamais qu'un test passe
si tu ne l'as pas lancé ; si quelque chose échoue, dis-le.
  5. Métriques obtenues (si applicable), avec le jeu de données utilisé.
  6. Limites, dette technique, risques.
  7. Ce que je dois vérifier moi-même (pas à pas).
  8. Étape suivante recommandée.
Mets aussi à jour docs/ia/journal.md (une ligne : date, étape, résumé) et coche la tâche dans docs/plan.md.
Termine en affichant le résumé du rapport dans le chat et propose un message de commit.
```

3 Service ai et couche LLM hybride

Socle technique du moteur | Durée indicative : 2 jours

Objectif

Créer le microservice ai et une abstraction LLM unique, avec Claude en mode démo et Ollama en secours local gratuit.

Pourquoi ce choix (regard d'expert)

Une interface *LLMProvider* découple le métier du fournisseur : on peut comparer les deux modèles sur le même jeu de test (argument scientifique fort) et la démo survit à une panne réseau. Journaliser latence, tokens et coût dès le départ permet de chiffrer le système dans le rapport.

Livrables

- services/ai/ complet (main.py, config, routes /health, Dockerfile, requirements)
- app/llm/ : base.py (interface), claude.py, ollama.py, fabrique + fallback
- Table llm_appels (provider, modele, tokens_entree, tokens_sortie, latence_ms, cout_estime, statut)
- Client catalog (ServiceClient) prêt pour les endpoints internes de l'étape 2-bis
- Gateway : préfixe /api/ai → ai:8008 ; docker-compose (image pgvector/pgvector:pg16, profil ollama) ; dev.ps1/stop-dev.ps1

> **v2 :** sans clé Anthropic, `LLM_PROVIDER=ollama` suffit pour tout le projet (coût nul).

> **Critère de « fini » :** GET /api/ai/health OK via le gateway ; un appel test fonctionne avec Ollama (et Claude si une clé est présente) ; le fallback se déclenche quand la clé Claude est absente.

Prompt à coller dans Claude Code — étape 3

Contexte : projet Voyara (dépôt actuel). Avant tout, lis CLAUDE.md, docs/architecture.md, docs/plan.md, docs/mcd.md, docs/ia/ et TOUS les rapports existants dans docs/ia/rapports/. Travaille en plan mode : propose d'abord ton plan (fichiers touchés, choix, risques) et attends ma validation avant d'écrire du code.
Contraintes permanentes : code, commentaires et messages en français ; champs métier en français (nom, prix, ...) ; découpage par domaine, aucune jointure SQL entre services (appels HTTP /internal/* avec INTERNAL_API_KEY via ServiceClient) ; réutiliser libs/common/voyara_common (config, auth, internal, database, pagination, exceptions, logging, limiter) ; textes d'interface via i18n fr/en/ar ; l'agent IA propose mais ne réserve ni ne paie jamais ; je suis sous Windows/PowerShell.

Étape 3 — Microservice `ai` + couche LLM hybride.

À faire :
1. Scaffold services/ai calqué sur services/catalog (structure app/, config via voyara_common.config, database, logging, limiter slowapi, Dockerfile, requirements). Port 8008, base voyara_ai. Active l'extension pgvector (passe l'image Postgres de docker-compose à pgvector/pgvector:pg16 et vérifie que les autres bases ne sont pas impactées).
2. Gateway : route le préfixe /api/ai vers le service ai (même mécanisme que les autres préfixes, blocage /internal conservé).
3. Couche LLM dans services/ai/app/llm/ :
   - base.py : interface LLMProvider avec generer(messages, systeme, outils=None, schema_sortie=None, max_tokens, temperature) → réponse normalisée (texte, appels_outils, tokens, latence).
   - claude.py : SDK officiel anthropic, modèle configurable (défaut claude-sonnet-5-5), tool use natif, sortie structurée.
   - ollama.py : API HTTP d'Ollama (modèle configurable, ex. qwen), mode JSON.
   - fabrique.py : choix par variable LLM_PROVIDER (claude | ollama | auto) ; en mode auto, Claude puis bascule sur Ollama en cas d'erreur/timeout ou de clé absente.
   Utilise le skill claude-api si disponible pour vérifier les identifiants de modèles et les paramètres.
4. Journalise chaque appel dans la table llm_appels (provider, modèle, tokens, latence_ms, coût estimé, statut, erreur).
5. Client interne vers catalog (ServiceClient de voyara_common) pour les endpoints de l'étape 2-bis (export hébergements, recherche vols/voitures/transferts) : timeouts, erreurs typées, aucun appel réel dans les tests.
6. docker-compose : service ai + service ollama sous un profil optionnel ; .env.example avec ANTHROPIC_API_KEY (vide), LLM_PROVIDER, OLLAMA_URL, OLLAMA_MODEL. Ajoute ai à scripts/dev.ps1 et stop-dev.ps1.
7. Tests pytest avec providers simulés (aucun appel réseau réel dans les tests) + un script services/ai/scripts/essai_llm.py pour un test manuel des deux providers.
Ne mets jamais de clé API dans le code ni dans le rapport.

RAPPORT OBLIGATOIRE — à la fin, crée docs/ia/rapports/etape-03-service-ai-llm-hybride.md avec :
  1. Objectif de l'étape.
  2. Ce que tu as fait : chaque fichier créé/modifié, avec son rôle en une ligne.
  3. Choix techniques : pour CHAQUE décision, l'option retenue, les alternatives envisagées et pourquoi tu les as écartées.
  4. Vérification : commandes PowerShell exécutées et sortie réelle (extraits). N'écris jamais qu'un test passe si tu ne l'as pas lancé ; si quelque chose échoue, dis-le.
  5. Métriques obtenues (si applicable), avec le jeu de données utilisé.
  6. Limites, dette technique, risques.
  7. Ce que je dois vérifier moi-même (pas à pas).
  8. Étape suivante recommandée.
Mets aussi à jour docs/ia/journal.md (une ligne : date, étape, résumé) et coche la tâche dans docs/plan.md.
Termine en affichant le résumé du rapport dans le chat et propose un message de commit.

4 Module 1 — Comprendre le voyageur (NLU)

Module 1 — Compréhension du besoin | Durée indicative : 3 jours

Objectif

Transformer « Je vais à Djerba 3 jours, 300 DT, en famille, culture et plage » — ou « On part de Tunis en avion, 4 jours à Djerba, 1200 DT à deux, avec une voiture » — en une demande structurée et validée.

Pourquoi ce choix (regard d'expert)

C'est de l'extraction d'information : on impose au LLM un schéma (sortie structurée / tool use) au lieu de parser du texte libre, ce qui supprime la majorité des erreurs de format. Un jeu de test annoté (fr, en, ar, darija) et un F1 par champ transforment une démo en résultat mesurable. Quand une information manque — budget, ville de départ d'un vol — le système pose une question au lieu d'inventer.

Livrables

- Schéma Pydantic DemandeVoyage (destination, date_debut, duree_jours, budget_total_dt, nb_adultes, nb_enfants, type_voyage, preferences, contraintes, niveau_marche_max, langue **+ ville_depart, besoins_transport, preferences_transport, budget_inclut_transport**)
- POST /api/ai/comprendre → demande + champs_manquants + question_clarification
- data/eval/nlu_demandes.jsonl : 100 demandes annotées (≥ 20 en arabe/darija, ≥ 25 avec transport)
- scripts/evaluer_nlu.py : exactitude et F1 par champ, par langue et par provider

> **v2 :** bloc transport dans le schéma, règles de normalisation transport, question de clarification si un vol est demandé sans ville de départ ou sans date.

> **Critère de « fini » :** F1 moyen ≥ 0,85 sur le jeu de test (avec le provider utilisé) ; champs transport ≥ 0,85 ; tableau comparatif Claude vs Ollama dans le rapport si les deux sont disponibles.

Prompt à coller dans Claude Code — étape 4

Contexte : projet Voyara (dépôt actuel). Avant tout, lis CLAUDE.md, docs/architecture.md, docs/plan.md, docs/mcd.md, docs/ia/ et TOUS les rapports existants dans docs/ia/rapports/. Travaille en plan mode : propose d'abord ton plan (fichiers touchés, choix, risques) et attends ma validation avant d'écrire du code.
Contraintes permanentes : code, commentaires et messages en français ; champs métier en français (nom, prix, ...) ; découpage par domaine, aucune jointure SQL entre services (appels HTTP /internal/* avec INTERNAL_API_KEY via ServiceClient) ; réutiliser libs/common/voyara_common (config, auth, internal, database, pagination, exceptions, logging, limiter) ; textes d'interface via i18n fr/en/ar ; l'agent IA propose mais ne réserve ni ne paie jamais ; je suis sous Windows/PowerShell.

Étape 4 — Module 1 : compréhension de la demande en langage naturel.

À faire dans services/ai :
1. Schéma Pydantic DemandeVoyage : destination (ville normalisée parmi les villes connues de catalog/pois, + texte brut), date_debut (optionnelle), duree_jours, budget_total_dt, nb_adultes, nb_enfants, type_voyage (famille | couple | solo | amis), preferences (thèmes de l'étape 2), contraintes (texte + champs : niveau_marche_max, mobilite_reduite, horaires), langue détectée.
   Bloc transport : ville_depart (optionnelle), besoins_transport (vol, voiture, transfert : chacun oui | non | indifferent ; défaut indifferent), preferences_transport (vol_direct, boite_auto, priorite : prix | confort), budget_inclut_transport (bool ; vrai par défaut si un vol est demandé). Validateurs (budget > 0, durée 1–14, ville_depart ≠ destination, etc.).
2. Service d'extraction : prompt système en français + sortie structurée (tool use côté Claude, mode JSON côté Ollama) via la couche LLM de l'étape 3. Normalisation post-LLM (« 300 dinars », « 300dt » → 300 ; « Jerba » → Djerba ; « 3 nuits » → 3 jours, justifie la règle ; « en avion depuis Tunis » → ville_depart = Tunis, vol = oui ; « on loue une voiture » → voiture = oui ; « sans voiture » → voiture = non).
3. Gestion des manques : renvoyer champs_manquants et une question_clarification dans la langue du voyageur ; ne jamais inventer un budget, une date ni une ville de départ. Si un vol est demandé sans ville_depart ou sans date_debut, poser la question.
4. Endpoint POST /api/ai/comprendre (auth optionnelle via get_current_identity, rate limit).
5. Jeu d'évaluation data/eval/nlu_demandes.jsonl : 100 demandes variées et annotées à la main (fautes, darija en lettres latines « 3 jours f Jerba », arabe, anglais, demandes incomplètes, demandes hors sujet), dont au moins 25 avec transport (vol, voiture, transfert, combinaisons, refus explicites). Propose-les, je les relirai.
6. scripts/evaluer_nlu.py : exactitude par champ, F1 sur les listes, matrice d'erreurs, résultats par langue, par provider, et séparément pour les champs transport ; temps et coût moyens ; sortie Markdown + CSV dans docs/ia/resultats/.
7. Tests pytest (normalisation, validateurs, provider simulé).

RAPPORT OBLIGATOIRE — à la fin, crée docs/ia/rapports/etape-04-module1-comprehension.md avec :
  1. Objectif de l'étape.
  2. Ce que tu as fait : chaque fichier créé/modifié, avec son rôle en une ligne.
  3. Choix techniques : pour CHAQUE décision, l'option retenue, les alternatives envisagées et pourquoi tu les as écartées.
  4. Vérification : commandes PowerShell exécutées et sortie réelle (extraits). N'écris jamais qu'un test passe si tu ne l'as pas lancé ; si quelque chose échoue, dis-le.
  5. Métriques obtenues (si applicable), avec le jeu de données utilisé.
  6. Limites, dette technique, risques.
  7. Ce que je dois vérifier moi-même (pas à pas).
  8. Étape suivante recommandée.
Mets aussi à jour docs/ia/journal.md (une ligne : date, étape, résumé) et coche la tâche dans docs/plan.md.
Termine en affichant le résumé du rapport dans le chat et propose un message de commit.

5 Module 2 — Recherche sémantique, RAG et recherche transport

Module 2 — Recherche & collecte | Durée indicative : 3–4 jours

Objectif

Retrouver les activités, hébergements et POI pertinents même quand les mots diffèrent (« découvrir le patrimoine » ↔ « visite de la Ghriba »), et trouver les vols, voitures et transferts disponibles qui correspondent exactement à la demande.

Pourquoi ce choix (regard d'expert)

Les embeddings capturent le sens, BM25 capture les noms propres : la recherche **hybride** (fusion RRF) bat chacune des deux seules, et on le prouve avec Recall@k et nDCG. Un modèle multilingue local (bge-m3 ou multilingual-e5) gère fr/en/ar sans coût d'API. Pour le transport, le bon outil n'est **pas** la similarité mais le filtre exact sur une donnée fraîche : deux moteurs de recherche, chacun adapté à sa donnée.

Livrables

- Table documents_index (source, source_id, type, ville, lat/lng, prix, texte, embedding vector, maj_le)
- Pipeline d'indexation incrémental depuis /internal/activities/export + export hébergements (2-bis) + pois
- POST /api/ai/rechercher (requête + filtres) → résultats avec score détaillé
- **app/transport/ + POST /api/ai/transport/rechercher** : recherche structurée vols/voitures/transferts via catalog
- data/eval/recherche_requetes.jsonl + scripts/evaluer_recherche.py (Recall@5/10, nDCG@10, MRR)

> **Critère de « fini » :** Hybride > vecteur seul > BM25 seul sur le jeu de test, chiffres dans le rapport ; réindexation < 2 min ; recherche transport exacte à 100 % sur le jeu de requêtes structurées.

Prompt à coller dans Claude Code — étape 5

Contexte : projet Voyara (dépôt actuel). Avant tout, lis CLAUDE.md, docs/architecture.md, docs/plan.md, docs/mcd.md, docs/ia/ et TOUS les rapports existants dans docs/ia/rapports/. Travaille en plan mode : propose d'abord ton plan (fichiers touchés, choix, risques) et attends ma validation avant d'écrire du code.
Contraintes permanentes : code, commentaires et messages en français ; champs métier en français (nom, prix, ...) ; découpage par domaine, aucune jointure SQL entre services (appels HTTP /internal/* avec INTERNAL_API_KEY via ServiceClient) ; réutiliser libs/common/voyara_common (config, auth, internal, database, pagination, exceptions, logging, limiter) ; textes d'interface via i18n fr/en/ar ; l'agent IA propose mais ne réserve ni ne paie jamais ; je suis sous Windows/PowerShell.

Étape 5 — Module 2 : recherche sémantique hybride (RAG) et recherche structurée du transport.

À faire dans services/ai :
1. Compare brièvement 2–3 modèles d'embeddings multilingues exécutables en local (ex. BAAI/bge-m3, intfloat/multilingual-e5-base) sur un mini-jeu fr/en/ar, choisis-en un et justifie (qualité, taille, vitesse CPU sous Windows).
2. Table documents_index (pgvector) : source (activite | hebergement | poi), source_id, type, ville, latitude, longitude, prix, texte_indexe (titre + description + tags, dans les trois langues si disponibles), embedding, empreinte (hash pour réindexation incrémentale), maj_le. Index HNSW. Les vols, voitures et transferts ne sont PAS indexés (voir point 8).
3. Pipeline scripts/indexer.py + endpoint interne /internal/reindexer : récupère activités (/internal/activities/export), hébergements (export interne de l'étape 2-bis) et POI, ne recalcule que ce qui a changé.
4. Recherche hybride : filtres durs (ville, rayon en km autour d'un point via haversine, prix max, type), puis BM25 (PostgreSQL full-text ou rank_bm25) + similarité cosinus, fusion Reciprocal Rank Fusion. Renvoie pour chaque résultat les scores partiels (explicabilité).
5. Endpoint POST /api/ai/rechercher.
6. Évaluation : data/eval/recherche_requetes.jsonl (≥ 40 requêtes avec documents pertinents annotés), scripts/evaluer_recherche.py comparant BM25 seul / vecteur seul / hybride : Recall@5, Recall@10, nDCG@10, MRR, latence p50/p95 → docs/ia/resultats/.
7. Tests pytest (haversine, RRF, filtres).
8. Transport (recherche structurée, sans embeddings) : services/ai/app/transport/ appelle les endpoints internes de catalog de l'étape 2-bis (vols, voitures, transferts) à partir d'une DemandeVoyage (ville_depart, destination, dates dérivées de date_debut + duree_jours, passagers = adultes + enfants), avec timeouts, cache court (5 min) et dégradation propre si catalog est indisponible (liste vide + avertissement, jamais d'offre inventée). Endpoint POST /api/ai/transport/rechercher (type, critères). Jeu data/eval/transport_requetes.jsonl (≥ 20 requêtes) vérifiant que chaque résultat respecte les filtres. Explique dans le rapport pourquoi le transport n'est pas indexé en vecteurs.

RAPPORT OBLIGATOIRE — à la fin, crée docs/ia/rapports/etape-05-module2-recherche-semantique.md avec :
  1. Objectif de l'étape.
  2. Ce que tu as fait : chaque fichier créé/modifié, avec son rôle en une ligne.
  3. Choix techniques : pour CHAQUE décision, l'option retenue, les alternatives envisagées et pourquoi tu les as écartées.
  4. Vérification : commandes PowerShell exécutées et sortie réelle (extraits). N'écris jamais qu'un test passe si tu ne l'as pas lancé ; si quelque chose échoue, dis-le.
  5. Métriques obtenues (si applicable), avec le jeu de données utilisé.
  6. Limites, dette technique, risques.
  7. Ce que je dois vérifier moi-même (pas à pas).
  8. Étape suivante recommandée.
Mets aussi à jour docs/ia/journal.md (une ligne : date, étape, résumé) et coche la tâche dans docs/plan.md.
Termine en affichant le résumé du rapport dans le chat et propose un message de commit.

6 Module 3 — Recommandation Top-5 et Top-3 transport

Module 3 — Recommandation personnalisée | Durée indicative : 3 jours

Objectif

Passer de 50 résultats pertinents aux 5 expériences qui correspondent vraiment au voyageur (goûts, budget, localisation, profil, temps disponible), et aux 3 meilleures options de vol, de voiture ou de transfert quand il en demande.

Pourquoi ce choix (regard d'expert)

On choisit un **score hybride explicable** (content-based + signaux de popularité + historique) plutôt qu'un filtrage collaboratif : avec peu d'utilisateurs, le collaboratif souffre du démarrage à froid. Chaque composante du score sert ensuite de justification à l'agent. MMR évite de proposer cinq plages identiques. Le transport suit la même logique : un score multicritère (prix, durée, escales, horaires) dont chaque composante est une raison lisible.

Livrables

- app/recommandation.py : score = Σ poids × (pertinence, budget, distance, profil, note, popularité, favoris)
- **app/transport/classement.py : Top-3 par type (prix, durée, escales, compatibilité horaire, capacité)**
- Poids configurables (table parametres_ia) + re-classement MMR
- Appels /internal/* vers review (notes, favoris) et booking (historique) — à créer s'ils manquent
- POST /api/ai/recommander → Top-N avec raisons + options de transport ; évaluation hors-ligne

> **Critère de « fini » :** Top-5 et Top-3 transport renvoyés en < 1 s avec raisons lisibles ; rapport avec ablation (effet de chaque composante).

Prompt à coller dans Claude Code — étape 6

Contexte : projet Voyara (dépôt actuel). Avant tout, lis CLAUDE.md, docs/architecture.md, docs/plan.md, docs/mcd.md, docs/ia/ et TOUS les rapports existants dans docs/ia/rapports/. Travaille en plan mode : propose d'abord ton plan (fichiers touchés, choix, risques) et attends ma validation avant d'écrire du code.
Contraintes permanentes : code, commentaires et messages en français ; champs métier en français (nom, prix, ...) ; découpage par domaine, aucune jointure SQL entre services (appels HTTP /internal/* avec INTERNAL_API_KEY via ServiceClient) ; réutiliser libs/common/voyara_common (config, auth, internal, database, pagination, exceptions, logging, limiter) ; textes d'interface via i18n fr/en/ar ; l'agent IA propose mais ne réserve ni ne paie jamais ; je suis sous Windows/PowerShell.

Étape 6 — Module 3 : recommandation personnalisée Top-5 et classement du transport.

À faire :
1. Dans services/ai/app/recommandation.py, un score explicable par candidat (issu de la recherche de l'étape 5) : pertinence_semantique, adequation_budget (prix / budget restant par jour et par personne, budget transport déduit si budget_inclut_transport), proximite (distance à l'hébergement ou au centre de la ville), adequation_profil (public famille/enfants, niveau_marche ≤ niveau_marche_max, thèmes), note_moyenne et nb_avis (lissage bayésien), popularite (nb réservations), bonus_favoris_historique. Normalise chaque composante dans [0,1].
2. Poids stockés dans une table parametres_ia (modifiables depuis l'admin à l'étape 10), valeurs par défaut justifiées.
3. Diversité : re-classement MMR (lambda configurable) pour éviter les doublons de thème/lieu.
4. Données externes : si review et booking n'exposent pas déjà des endpoints /internal/*/batch utiles (notes moyennes par activité, favoris d'un utilisateur, activités réservées), ajoute-les dans ces services, protégés par require_internal_key, avec appels groupés (asyncio.gather) et timeouts. Si un service est indisponible, la recommandation se dégrade proprement.
5. Chaque recommandation renvoie raisons : liste de phrases générées à partir des composantes dominantes (pas par le LLM), en fr/en/ar.
6. Transport : services/ai/app/transport/classement.py classe les offres de l'étape 5 par type demandé (vol, voiture, transfert) avec un score explicable : prix (normalisé vs budget), durée totale, nombre d'escales, compatibilité horaire (arrivée pas trop tardive, départ pas trop matinal), capacité/bagages vs groupe, note si disponible ; respect des preferences_transport (vol_direct, boite_auto, priorite prix | confort). Top-3 par type, avec raisons fr/en/ar. Poids dans parametres_ia.
7. Endpoint POST /api/ai/recommander (DemandeVoyage en entrée) → recommandations + transport_options.
8. Évaluation scripts/evaluer_reco.py : précision@5 sur des profils synthétiques annotés, couverture du catalogue, diversité intra-liste, ablation (score sans chaque composante), et pour le transport : taux de Top-1 conforme aux préférences sur 20 cas annotés. Résultats dans docs/ia/resultats/.

RAPPORT OBLIGATOIRE — à la fin, crée docs/ia/rapports/etape-06-module3-recommandation.md avec :
  1. Objectif de l'étape.
  2. Ce que tu as fait : chaque fichier créé/modifié, avec son rôle en une ligne.
  3. Choix techniques : pour CHAQUE décision, l'option retenue, les alternatives envisagées et pourquoi tu les as écartées.
  4. Vérification : commandes PowerShell exécutées et sortie réelle (extraits). N'écris jamais qu'un test passe si tu ne l'as pas lancé ; si quelque chose échoue, dis-le.
  5. Métriques obtenues (si applicable), avec le jeu de données utilisé.
  6. Limites, dette technique, risques.
  7. Ce que je dois vérifier moi-même (pas à pas).
  8. Étape suivante recommandée.
Mets aussi à jour docs/ia/journal.md (une ligne : date, étape, résumé) et coche la tâche dans docs/plan.md.
Termine en affichant le résumé du rapport dans le chat et propose un message de commit.

7 Modules 4 & 5 — Itinéraire et optimisation sous contraintes

Module 4 — Itinéraire · Module 5 — Budget & optimisation | Durée indicative : 4–5 jours

Objectif

Construire un planning jour par jour (matin, midi, après-midi, soir) réalisable, de l'arrivée au départ : vols, transfert aéroport ↔ hébergement ou voiture de location, budget total, temps, distances, horaires, places disponibles, repas, hébergement.

Pourquoi ce choix (regard d'expert)

C'est un problème d'optimisation combinatoire, pas un problème de langage : le LLM ne doit pas « inventer » l'itinéraire. On le modélise avec **OR-Tools CP-SAT** (variables d'affectation activité→créneau, choix de l'option de vol et du mode de mobilité, contraintes dures de budget/horaires/compatibilité, objectif = somme des scores du module 3 moins pénalité de distance), avec un algorithme glouton en secours. Le résultat est garanti cohérent : « un voyage réalisable, pas seulement idéal ».

Livrables

- app/itineraire/ : modele.py (Itineraire, Jour, Creneau, Etape, Budget), solveur_cpsat.py, glouton.py
- Étapes de type **vol, transfert, location_voiture** en plus de activite, repas, detente, trajet
- Ventilation du budget : **vols, transferts, location voiture**, hébergement, repas, activités, déplacements
- Vérification des créneaux réels, des places de vol et de la météo (activités intérieures si pluie)
- POST /api/ai/itineraire + tests de propriétés (hypothesis)

> **Critère de « fini » :** scénario Djerba/3 jours/300 DT/famille (sans transport) et scénario Tunis→Djerba en avion/4 jours/1200 DT/couple avec voiture : itinéraires valides en < 3 s, total ≤ budget, aucune activité avant l'arrivée ni après le départ ; tests de propriétés verts.

Prompt à coller dans Claude Code — étape 7

Contexte : projet Voyara (dépôt actuel). Avant tout, lis CLAUDE.md, docs/architecture.md, docs/plan.md, docs/mcd.md, docs/ia/ et TOUS les rapports existants dans docs/ia/rapports/. Travaille en plan mode : propose d'abord ton plan (fichiers touchés, choix, risques) et attends ma validation avant d'écrire du code. Contraintes permanentes : code, commentaires et messages en français ; champs métier en français (nom, prix, ...) ; découpage par domaine, aucune jointure SQL entre services (appels HTTP /internal/* avec INTERNAL_API_KEY via ServiceClient) ; réutiliser libs/common/voyara_common (config, auth, internal, database, pagination, exceptions, logging, limiter) ; textes d'interface via i18n fr/en/ar ; l'agent IA propose mais ne réserve ni ne paie jamais ; je suis sous Windows/PowerShell.

Étape 7 — Modules 4 et 5 : construction de l'itinéraire et optimisation du budget, transport compris.

À faire dans services/ai/app/itineraire/ :
1. Modèles Pydantic : Itineraire (demande, jours, transport, budget, score_global, avertissements), Jour (date, creneaux), Creneau (matin | midi | apres_midi | soir), Etape (type activite | repas | detente | trajet | vol | transfert | location_voiture, source, source_id, heure_debut, duree_min, cout_estime_dt, distance_depuis_precedent_km, raisons).
2. Budget : vols (prix total pour tous les passagers, aller + retour), transferts, location voiture (prix total + carburant par km configurable), hébergement (prix_nuit × nuits, capacité compatible avec le groupe), repas (prix_moyen des POI restaurants ou forfait par personne configurable), activités (prix × nb personnes, tarif enfant si disponible), déplacements sur place (taxi par km si pas de voiture). Ventilation renvoyée comme dans la présentation : Vols / Transferts / Location voiture / Hébergement / Restaurants / Activités / Déplacements / Total. Si budget_inclut_transport est faux, le transport est affiché à part et exclu de la contrainte.
3. Solveur OR-Tools CP-SAT : variables binaires activité×jour×créneau ; variables de choix parmi le Top-3 de l'étape 6 : une option de vol aller et une retour (si vol demandé), un mode de mobilité (voiture | transferts + taxi, si indifferent). Contraintes dures : au plus une activité par créneau, durée compatible, moment_ideal, horaires d'ouverture, budget total, chaque activité au plus une fois, places disponibles sur la date (appel /internal/availabilities ou export), niveau_marche ≤ max, activités intérieures privilégiées si la météo prévoit de la pluie ; aucune activité avant arrivée du vol + transfert + marge (configurable, ex. 90 min) ni après départ − marge ; transfert aéroport → hébergement le premier jour et retour le dernier jour (ou prise/restitution de la voiture). Objectif : maximiser Σ score_reco − α × distance totale − β × écart d'équilibre entre les jours − γ × coût transport. Limite de temps 2 s. Repas placés au midi et au soir près de l'étape précédente.
4. Glouton de secours si CP-SAT échoue ou dépasse le temps ; le rapport compare qualité et temps des deux.
5. Si aucune solution n'existe (budget trop faible), renvoyer un avertissement clair et la meilleure solution relâchée (vol moins cher ou date ± 1 jour, transferts au lieu de la voiture, moins d'activités payantes) au lieu d'une erreur.
6. Endpoint POST /api/ai/itineraire (DemandeVoyage + options).
7. Tests pytest + hypothesis : total ≤ budget, pas de chevauchement horaire, chaque activité unique, distances cohérentes, aucune activité hors de la fenêtre arrivée/départ. Cas de référence : Djerba 3 jours 300 DT famille culture+plage (sans transport) et Tunis→Djerba en avion, 4 jours, 1200 DT, couple, voiture.

RAPPORT OBLIGATOIRE — à la fin, crée docs/ia/rapports/etape-07-modules4-5-itineraire-budget.md avec :
  1. Objectif de l'étape.
  2. Ce que tu as fait : chaque fichier créé/modifié, avec son rôle en une ligne.
  3. Choix techniques : pour CHAQUE décision, l'option retenue, les alternatives envisagées et pourquoi tu les as écartées.
  4. Vérification : commandes PowerShell exécutées et sortie réelle (extraits). N'écris jamais qu'un test passe si tu ne l'as pas lancé ; si quelque chose échoue, dis-le.
  5. Métriques obtenues (si applicable), avec le jeu de données utilisé.
  6. Limites, dette technique, risques.
  7. Ce que je dois vérifier moi-même (pas à pas).
  8. Étape suivante recommandée.
Mets aussi à jour docs/ia/journal.md (une ligne : date, étape, résumé) et coche la tâche dans docs/plan.md.
Termine en affichant le résumé du rapport dans le chat et propose un message de commit.

8 Module 6 — Agent intelligent

Module 6 — Adaptation intelligente | Durée indicative : 3–4 jours

Objectif

Orchestrer les modules dans une conversation : planifier, modifier (« je préfère moins marcher », « finalement pas de voiture »), justifier (« pourquoi cette activité ? », « pourquoi ce vol ? »).

Pourquoi ce choix (regard d'expert)

L'agent est un LLM qui **choisit des outils** (nos modules) au lieu de répondre de mémoire : les faits viennent de la base, les calculs du solveur, le LLM ne fait que comprendre et expliquer. C'est ce qui limite les hallucinations — un prix ou un horaire de vol n'est jamais inventé. L'état de la conversation est persistant pour que les modifications soient incrémentales. L'agent propose ; il ne réserve et ne paie jamais seul.

Livrables

- app/agent/ : boucle à outils (analyser_demande, rechercher, **rechercher_transport**, recommander, construire_itineraire, modifier_itineraire, justifier_choix, meteo, proposer_reservation), garde-fous (max itérations, budget tokens)
- Tables conversations, messages, itineraires_versions
- POST /api/ai/agent/messages en streaming SSE
- Tests des 4 scénarios (planifier, modifier, justifier, **transport**)

> **Critère de « fini » :** les 4 scénarios passent de bout en bout (Ollama, et Claude si une clé est présente) ; chaque justification correspond à des valeurs réelles du score.

Prompt à coller dans Claude Code — étape 8

Contexte : projet Voyara (dépôt actuel). Avant tout, lis CLAUDE.md, docs/architecture.md, docs/plan.md, docs/mcd.md, docs/ia/ et TOUS les rapports existants dans docs/ia/rapports/. Travaille en plan mode : propose d'abord ton plan (fichiers touchés, choix, risques) et attends ma validation avant d'écrire du code.
Contraintes permanentes : code, commentaires et messages en français ; champs métier en français (nom, prix, ...) ; découpage par domaine, aucune jointure SQL entre services (appels HTTP /internal/* avec INTERNAL_API_KEY via ServiceClient) ; réutiliser libs/common/voyara_common (config, auth, internal, database, pagination, exceptions, logging, limiter) ; textes d'interface via i18n fr/en/ar ; l'agent IA propose mais ne réserve ni ne paie jamais ; je suis sous Windows/PowerShell.

Étape 8 — Module 6 : agent conversationnel à outils.

À faire dans services/ai/app/agent/ :
1. Définis les outils (schémas JSON) qui appellent les modules existants, sans dupliquer leur logique : analyser_demande (étape 4), rechercher (5), rechercher_transport (5, type vol | voiture | transfert), recommander (6, avec Top-3 transport), construire_itineraire et modifier_itineraire (7), justifier_choix (renvoie les composantes de score d'une étape, y compris d'une option de transport), obtenir_meteo (1), proposer_reservation (type activite | hebergement | vol | voiture | transfert : renvoie seulement un lien vers la page correspondante du site, ne crée rien).
2. Boucle agentique via la couche LLM (tool use Claude ; pour Ollama, outils en mode JSON) : max 8 itérations, budget de tokens par tour, journal de chaque appel d'outil. Prompt système en français : rôle, règles (n'affirme aucun prix, horaire, place ou numéro de vol qui ne vient pas d'un outil, réponds dans la langue de l'utilisateur, demande une précision si nécessaire, ne réserve jamais, ne demande jamais de numéro de passeport, de permis ou de carte bancaire).
3. Persistance : tables conversations (utilisateur_id nullable, langue, cree_le), messages, itineraires_versions (JSON + version) ; la modification part toujours de la dernière version.
4. Modification : « je préfère moins marcher » → niveau_marche_max diminue, remplacement des activités concernées, recalcul des distances et du planning ; « finalement pas de voiture » → mode transferts + taxi, recalcul du budget ; « un vol plus tôt » → autre option du Top-3. Réponse qui liste les changements (diff entre versions).
5. Justification : « pourquoi Guellala ? » ou « pourquoi ce vol ? » → réponse construite à partir de justifier_choix (budget respecté, distance, thème, durée, escales…).
6. Endpoint POST /api/ai/agent/messages en Server-Sent Events (texte en streaming + événements itineraire_mis_a_jour), GET /api/ai/agent/conversations/{id}. Auth optionnelle, rate limit plus strict.
7. Tests : les 4 scénarios (planifier ; modifier « moins marcher » ; justifier ; transport « ajoute le vol depuis Tunis et une voiture » puis « finalement des transferts ») avec un provider simulé déterministe, + un script manuel scripts/scenario_demo.py qui les joue avec le vrai provider et enregistre la transcription dans docs/ia/resultats/.

RAPPORT OBLIGATOIRE — à la fin, crée docs/ia/rapports/etape-08-module6-agent.md avec :
  1. Objectif de l'étape.
  2. Ce que tu as fait : chaque fichier créé/modifié, avec son rôle en une ligne.
  3. Choix techniques : pour CHAQUE décision, l'option retenue, les alternatives envisagées et pourquoi tu les as écartées.
  4. Vérification : commandes PowerShell exécutées et sortie réelle (extraits). N'écris jamais qu'un test passe si tu ne l'as pas lancé ; si quelque chose échoue, dis-le.
  5. Métriques obtenues (si applicable), avec le jeu de données utilisé.
  6. Limites, dette technique, risques.
  7. Ce que je dois vérifier moi-même (pas à pas).
  8. Étape suivante recommandée.
Mets aussi à jour docs/ia/journal.md (une ligne : date, étape, résumé) et coche la tâche dans docs/plan.md.
Termine en affichant le résumé du rapport dans le chat et propose un message de commit.

9 Interface voyageur : « Planifier mon voyage »

Interface Voyageur | Durée indicative : 3 jours

Objectif

Donner vie au moteur côté client : champ « Décrivez votre voyage… », chat, itinéraire avec vols et transferts, carte, budget, réservation, partage.

Pourquoi ce choix (regard d'expert)

Le jury jugera d'abord ce qu'il voit. Le streaming rend l'agent vivant, la carte rend la cohérence géographique évidente, les boutons Réserver relient l'IA au cœur métier existant (booking + Stripe) pour chaque élément — activité, hébergement, vol, voiture, transfert : c'est la vision globale « de la planification à la réalisation ».

Livrables

- frontend/apps/client : page /planifier, composants Chat, TimelineJour (avec étapes vol/transfert/voiture), CarteItineraire (aéroport + trajets), BudgetVentilation (nouveaux postes)
- Client API dans @voyara/api (SSE) ; textes fr/en/ar + RTL dans @voyara/i18n
- Partage par lien + export PDF/impression de l'itinéraire
- Feedback 👍 / 👎 par itinéraire (envoyé au service ai)

> **Critère de « fini » :** scénario Djerba et scénario transport joués entièrement dans le navigateur en fr et en ar (RTL correct), sur mobile et desktop.

Prompt à coller dans Claude Code — étape 9

Contexte : projet Voyara (dépôt actuel). Avant tout, lis CLAUDE.md, docs/architecture.md, docs/plan.md, docs/mcd.md, docs/ia/ et TOUS les rapports existants dans docs/ia/rapports/. Travaille en plan mode : propose d'abord ton plan (fichiers touchés, choix, risques) et attends ma validation avant d'écrire du code.
Contraintes permanentes : code, commentaires et messages en français ; champs métier en français (nom, prix, ...) ; découpage par domaine, aucune jointure SQL entre services (appels HTTP /internal/* avec INTERNAL_API_KEY via ServiceClient) ; réutiliser libs/common/voyara_common (config, auth, internal, database, pagination, exceptions, logging, limiter) ; textes d'interface via i18n fr/en/ar ; l'agent IA propose mais ne réserve ni ne paie jamais ; je suis sous Windows/PowerShell.

Étape 9 — Interface voyageur « Planifier mon voyage » (Next.js, apps/client).

À faire :
1. Page /planifier (App Router ; page serveur pour le SEO + composant client pour le chat), lien dans la navbar et sur la page d'accueil (« Décrivez votre voyage… » + bouton « Générer mon voyage »), suggestions d'exemples (dont un exemple avec vol depuis Tunis).
2. Client SSE dans @voyara/api (compatible Vite et Next, refresh du token conservé) pour POST /api/ai/agent/messages.
3. Composants dans le style existant (Tailwind v4, palette sable/encre/sarcelle/terracotta/or, Marcellus/Albert Sans, @voyara/ui) : Chat (streaming, indicateur « l'agent recherche… » par outil), TimelineJour (matin/midi/après-midi/soir, coût, distance, raisons dépliables ; étapes vol, transfert et location de voiture avec horaires et icônes), CarteItineraire (réutilise l'intégration MapLibre/OpenFreeMap existante, tracé par jour, aéroport et trajet de transfert), BudgetVentilation (barres par poste vs budget : vols, transferts, location voiture, hébergement, restaurants, activités, déplacements), bouton Réserver par élément vers la page existante (activité/créneau, hébergement, vol, voiture, transfert).
4. Partage : lien public en lecture seule (/planifier/[id]) et impression/export PDF propre (CSS print).
5. Feedback 👍/👎 + commentaire, envoyé à un endpoint POST /api/ai/feedback (à créer).
6. Toutes les chaînes dans @voyara/i18n (fr/en/ar), vérifie le RTL en arabe ; états vides, erreurs (ex. aucun vol disponible), chargement ; accessibilité clavier.
7. Vérifie dans le navigateur (dev server) le scénario Djerba puis le scénario transport, en fr puis en ar, et ajoute des captures dans docs/ia/figures/.

RAPPORT OBLIGATOIRE — à la fin, crée docs/ia/rapports/etape-09-interface-voyageur.md avec :
  1. Objectif de l'étape.
  2. Ce que tu as fait : chaque fichier créé/modifié, avec son rôle en une ligne.
  3. Choix techniques : pour CHAQUE décision, l'option retenue, les alternatives envisagées et pourquoi tu les as écartées.
  4. Vérification : commandes PowerShell exécutées et sortie réelle (extraits). N'écris jamais qu'un test passe si tu ne l'as pas lancé ; si quelque chose échoue, dis-le.
  5. Métriques obtenues (si applicable), avec le jeu de données utilisé.
  6. Limites, dette technique, risques.
  7. Ce que je dois vérifier moi-même (pas à pas).
  8. Étape suivante recommandée.
Mets aussi à jour docs/ia/journal.md (une ligne : date, étape, résumé) et coche la tâche dans docs/plan.md.
Termine en affichant le résumé du rapport dans le chat et propose un message de commit.

10 Back-office : paramétrage et analytics IA

Back-office VOYARA (Paramétrage IA, Analytics) Durée indicative : 2 jours

Objectif

Permettre à l'administrateur de piloter le moteur : fournisseur LLM, poids du scoring (activités et transport), paramètres de transport, validation des tags, suivi de l'usage et de la qualité.

Pourquoi ce choix (regard d'expert)

Un système d'IA en production se surveille : coût, latence, taux d'erreur, satisfaction. Le BFF admin existant agrège déjà les services avec asyncio.gather : on suit ce patron.

Livrables

- Endpoints admin du service ai (require_admin) : paramètres, statistiques, conversations anonymisées
- Agrégation dans le BFF admin (services/admin)
- Écrans Vite : Paramétrage IA (dont transport), Analytics IA (graphiques), Qualité (feedback)

> **Critère de « fini » :** changer un poids dans l'admin modifie immédiatement le classement ; tableau de bord affiché avec données réelles.

Prompt à coller dans Claude Code — étape 10

Contexte : projet Voyara (dépôt actuel). Avant tout, lis CLAUDE.md, docs/architecture.md, docs/plan.md, docs/mcd.md, docs/ia/ et TOUS les rapports existants dans docs/ia/rapports/. Travaille en plan mode : propose d'abord ton plan (fichiers touchés, choix, risques) et attends ma validation avant d'écrire du code.
Contraintes permanentes : code, commentaires et messages en français ; champs métier en français (nom, prix, ...) ; découpage par domaine, aucune jointure SQL entre services (appels HTTP /internal/* avec INTERNAL_API_KEY via ServiceClient) ; réutiliser libs/common/voyara_common (config, auth, internal, database, pagination, exceptions, logging, limiter) ; textes d'interface via i18n fr/en/ar ; l'agent IA propose mais ne réserve ni ne paie jamais ; je suis sous Windows/PowerShell.

Étape 10 — Back-office IA (apps/admin + BFF admin).

À faire :
1. Service ai : endpoints protégés par require_admin : GET/PUT /parametres (provider, modèle, température, poids du scoring des activités et du transport, lambda MMR, coûts unitaires repas/km, coût taxi/km, carburant/km, marge aéroport en minutes, tolérance de date des vols), GET /statistiques (requêtes/jour, latence p50/p95, tokens et coût par provider, taux de fallback, taux d'échec du solveur, part des itinéraires avec transport, taux d'offres transport introuvables, taux de 👍), GET /conversations (anonymisées, paginées via voyara_common.pagination).
2. BFF services/admin : agrège ces statistiques avec celles des réservations (taux de conversion itinéraire → réservation, par type : activité, hébergement, vol, voiture, transfert) via asyncio.gather et les endpoints /internal/*/batch, sur le modèle existant.
3. apps/admin : pages « Paramétrage IA » (formulaire validé, section transport, aperçu de l'effet sur une demande test), « Analytics IA » (graphiques), « Qualité » (feedbacks, conversations), plus l'écran de validation des tags de l'étape 2 s'il n'est pas déjà intégré. i18n fr/en/ar.
4. Tests des endpoints (admin requis, valeurs bornées).

RAPPORT OBLIGATOIRE — à la fin, crée docs/ia/rapports/etape-10-back-office-ia.md avec :
  1. Objectif de l'étape.
  2. Ce que tu as fait : chaque fichier créé/modifié, avec son rôle en une ligne.
  3. Choix techniques : pour CHAQUE décision, l'option retenue, les alternatives envisagées et pourquoi tu les as écartées.
  4. Vérification : commandes PowerShell exécutées et sortie réelle (extraits). N'écris jamais qu'un test passe si tu ne l'as pas lancé ; si quelque chose échoue, dis-le.
  5. Métriques obtenues (si applicable), avec le jeu de données utilisé.
  6. Limites, dette technique, risques.
  7. Ce que je dois vérifier moi-même (pas à pas).
  8. Étape suivante recommandée.
Mets aussi à jour docs/ia/journal.md (une ligne : date, étape, résumé) et coche la tâche dans docs/plan.md.
Termine en affichant le résumé du rapport dans le chat et propose un message de commit.

11 Évaluation globale, tests et CI

Démarche Data Science | Durée indicative : 2 jours

Objectif

Prouver scientifiquement que le moteur fonctionne : métriques reproductibles par module et de bout en bout, tests automatisés, intégration continue.

Pourquoi ce choix (regard d'expert)

C'est ce qui distingue un projet de data science d'une démo. On évalue chaque module isolément (NLU, recherche, reco, transport, solveur), puis l'itinéraire complet : contraintes respectées (vérifiable automatiquement) et qualité perçue (LLM-as-judge avec grille, recoupé par une petite évaluation humaine).

Livrables

- scripts/evaluer_tout.py → docs/ia/resultats/synthese.md (tableaux pour le rapport)
- Évaluation bout en bout sur 30 scénarios (≥ 10 avec transport) : taux de faisabilité, respect budget, cohérence horaire vol ↔ activités, distance moyenne, diversité, note LLM-as-judge, note humaine (5 personnes)
- Comparaison Claude vs Ollama (qualité, latence, coût) si les deux sont disponibles
- Tests pytest du service ai dans la CI GitHub Actions

> **Critère de « fini » :** une seule commande régénère tous les chiffres du rapport ; CI verte.

Prompt à coller dans Claude Code — étape 11

Contexte : projet Voyara (dépôt actuel). Avant tout, lis CLAUDE.md, docs/architecture.md, docs/plan.md, docs/mcd.md, docs/ia/ et TOUS les rapports existants dans docs/ia/rapports/. Travaille en plan mode : propose d'abord ton plan (fichiers touchés, choix, risques) et attends ma validation avant d'écrire du code. Contraintes permanentes : code, commentaires et messages en français ; champs métier en français (nom, prix, ...) ; découpage par domaine, aucune jointure SQL entre services (appels HTTP /internal/* avec INTERNAL_API_KEY via ServiceClient) ; réutiliser libs/common/voyara_common (config, auth, internal, database, pagination, exceptions, logging, limiter) ; textes d'interface via i18n fr/en/ar ; l'agent IA propose mais ne réserve ni ne paie jamais ; je suis sous Windows/PowerShell.

Étape 11 — Évaluation globale, tests et CI.

À faire :
1. scripts/evaluer_tout.py (racine services/ai) : lance les évaluations des étapes 4, 5 (sémantique et transport), 6, 7 et une évaluation bout en bout, avec graines aléatoires fixées, et produit docs/ia/resultats/synthese.md + CSV.
2. Bout en bout : data/eval/scenarios.jsonl (30 demandes réalistes, dont budgets serrés, demandes en arabe et au moins 10 avec vol/voiture/transfert) ; métriques automatiques : taux de compréhension correcte, taux d'itinéraires faisables, respect du budget (transport inclus), cohérence horaire avec les vols (aucune activité hors fenêtre), taux d'offres transport trouvées, distance moyenne par jour, diversité des thèmes, temps de réponse, coût.
3. LLM-as-judge : grille notée 1–5 (pertinence vs demande, cohérence, réalisme, équilibre), juge = modèle différent de celui qui génère si possible ; documente les biais connus. Prépare aussi un formulaire d'évaluation humaine (5 évaluateurs) et la méthode pour calculer l'accord juge/humains.
4. Comparaison Claude vs Ollama sur tous les scénarios (qualité, latence, coût) avec un tableau et un graphique ; si une seule source est disponible, dis-le clairement.
5. CI GitHub Actions : tests pytest du service ai (providers simulés, catalog simulé, Postgres pgvector en service), lint ; l'évaluation complète reste manuelle (coût API).
6. Liste les résultats honnêtement, y compris les échecs et leurs causes.

RAPPORT OBLIGATOIRE — à la fin, crée docs/ia/rapports/etape-11-evaluation-qualite.md avec :
  1. Objectif de l'étape.
  2. Ce que tu as fait : chaque fichier créé/modifié, avec son rôle en une ligne.
  3. Choix techniques : pour CHAQUE décision, l'option retenue, les alternatives envisagées et pourquoi tu les as écartées.
  4. Vérification : commandes PowerShell exécutées et sortie réelle (extraits). N'écris jamais qu'un test passe si tu ne l'as pas lancé ; si quelque chose échoue, dis-le.
  5. Métriques obtenues (si applicable), avec le jeu de données utilisé.
  6. Limites, dette technique, risques.
  7. Ce que je dois vérifier moi-même (pas à pas).
  8. Étape suivante recommandée.
Mets aussi à jour docs/ia/journal.md (une ligne : date, étape, résumé) et coche la tâche dans docs/plan.md. Termine en affichant le résumé du rapport dans le chat et propose un message de commit.

12 Sécurité, coûts et RGPD du moteur IA

Transverse | Durée indicative : 1–2 jours

Objectif

Rendre le moteur sûr et maîtrisé : injection de prompt, abus, coûts, données personnelles.

Pourquoi ce choix (regard d'expert)

Un agent connecté à des outils est une surface d'attaque. Les garde-fous (outils en lecture seule, aucune action irréversible, plafonds) et la conformité RGPD (la suppression par anonymisation existe déjà dans auth) sont attendus par un jury d'ingénieurs. Avec le transport, un risque nouveau apparaît : un agent qui demanderait un passeport ou un permis. Il ne doit jamais le faire.

Livrables

- Tests d'injection de prompt (jeu adversarial) et mesures correctives
- Plafonds : tokens par message, messages par jour et par IP/utilisateur, coût journalier max
- Masquage des PII dans les journaux ; durée de conservation des conversations ; aucun document d'identité collecté
- Extension de la suppression RGPD de auth aux conversations du service ai

> **Critère de « fini » :** jeu adversarial sans fuite du prompt système ni action non autorisée ; suppression RGPD testée.

Prompt à coller dans Claude Code — étape 12

Contexte : projet Voyara (dépôt actuel). Avant tout, lis CLAUDE.md, docs/architecture.md, docs/plan.md, docs/mcd.md, docs/ia/ et TOUS les rapports existants dans docs/ia/rapports/. Travaille en plan mode : propose d'abord ton plan (fichiers touchés, choix, risques) et attends ma validation avant d'écrire du code.
Contraintes permanentes : code, commentaires et messages en français ; champs métier en français (nom, prix, ...) ; découpage par domaine, aucune jointure SQL entre services (appels HTTP /internal/* avec INTERNAL_API_KEY via ServiceClient) ; réutiliser libs/common/voyara_common (config, auth, internal, database, pagination, exceptions, logging, limiter) ; textes d'interface via i18n fr/en/ar ; l'agent IA propose mais ne réserve ni ne paie jamais ; je suis sous Windows/PowerShell.

Étape 12 — Sécurité, maîtrise des coûts et RGPD du service ai.

À faire :
1. Revue de sécurité du service ai et de l'agent : vérifie que tous les outils sont en lecture seule ou sans effet irréversible (proposer_reservation ne renvoie qu'un lien, pour tous les types : activité, hébergement, vol, voiture, transfert), que les endpoints /internal/* restent inaccessibles via le gateway, que les entrées sont bornées (longueur des messages, taille des conversations).
2. Jeu adversarial data/eval/injections.jsonl (20+ tentatives : révéler le prompt système, ignorer les règles, faire réserver/payer un vol, injecter des instructions dans une description d'activité, d'hébergement, d'offre de transport ou de POI, obtenir un passeport ou un numéro de carte). Mesure le taux de résistance avant/après corrections.
3. Plafonds : rate limit slowapi par IP et par utilisateur, max tokens par tour, coût journalier maximal (au-delà : bascule automatique sur Ollama ou message d'indisponibilité), configurable.
4. Données personnelles : masquage des e-mails/téléphones/numéros de document dans les journaux et llm_appels, l'agent ne demande ni ne stocke aucun numéro de passeport, de permis ou de carte ; durée de conservation configurable des conversations anonymes, purge planifiée.
5. RGPD : quand un compte est supprimé (anonymisation existante dans auth), le service ai doit anonymiser ses conversations — ajoute l'appel interne et un test.
6. Ajoute une section « Limites connues du moteur IA » dans docs/architecture.md (dont : disponibilités transport vues à l'instant T, pas de garantie au moment de la réservation).

RAPPORT OBLIGATOIRE — à la fin, crée docs/ia/rapports/etape-12-securite-couts-rgpd.md avec :
  1. Objectif de l'étape.
  2. Ce que tu as fait : chaque fichier créé/modifié, avec son rôle en une ligne.
  3. Choix techniques : pour CHAQUE décision, l'option retenue, les alternatives envisagées et pourquoi tu les as écartées.
  4. Vérification : commandes PowerShell exécutées et sortie réelle (extraits). N'écris jamais qu'un test passe si tu ne l'as pas lancé ; si quelque chose échoue, dis-le.
  5. Métriques obtenues (si applicable), avec le jeu de données utilisé.
  6. Limites, dette technique, risques.
  7. Ce que je dois vérifier moi-même (pas à pas).
  8. Étape suivante recommandée.
Mets aussi à jour docs/ia/journal.md (une ligne : date, étape, résumé) et coche la tâche dans docs/plan.md.
Termine en affichant le résumé du rapport dans le chat et propose un message de commit.

13 Démo, rapport et soutenance

Livraison | Durée indicative : 2 jours

Objectif

Préparer une démonstration fiable des 4 scénarios et intégrer le moteur IA au rapport de PFE.

Pourquoi ce choix (regard d'expert)

Une démo en direct dépendant d'une API externe peut échouer : on prévoit un jeu de données figé, le mode Ollama hors-ligne et une vidéo de secours. Le rapport s'appuie sur les rapports d'étape et les chiffres générés à l'étape 11.

Livrables

- Script de seed de démo (activités Djerba taguées, POI, créneaux, **vols Tunis ↔ Djerba, voitures et transferts à Djerba**, comptes)
- docs/ia/demo.md : déroulé minuté des 4 scénarios + plan B
- Chapitre « Moteur IA » dans docs/rapport.tex (architecture, méthodes, résultats, limites)
- Diagrammes : architecture du service ai, séquence de l'agent, modèle de données voyara_ai

> **Critère de « fini » :** démo jouée 3 fois sans erreur, en ligne et hors-ligne ; chapitre du rapport relu.

Prompt à coller dans Claude Code — étape 13

Contexte : projet Voyara (dépôt actuel). Avant tout, lis CLAUDE.md, docs/architecture.md, docs/plan.md, docs/mcd.md, docs/ia/ et TOUS les rapports existants dans docs/ia/rapports/. Travaille en plan mode : propose d'abord ton plan (fichiers touchés, choix, risques) et attends ma validation avant d'écrire du code. Contraintes permanentes : code, commentaires et messages en français ; champs métier en français (nom, prix, ...) ; découpage par domaine, aucune jointure SQL entre services (appels HTTP /internal/* avec INTERNAL_API_KEY via ServiceClient) ; réutiliser libs/common/voyara_common (config, auth, internal, database, pagination, exceptions, logging, limiter) ; textes d'interface via i18n fr/en/ar ; l'agent IA propose mais ne réserve ni ne paie jamais ; je suis sous Windows/PowerShell.

Étape 13 — Démo et intégration au rapport de PFE.

À faire :
1. Script scripts/seed_demo_ia.py : données de démo cohérentes (Djerba : activités taguées avec créneaux sur les dates de soutenance, hébergements famille et couple, POI restaurants/plages, aéroport de Djerba-Zarzis, vols Tunis ↔ Djerba avec places, voitures de location et transferts aéroport ↔ hôtels), comptes de démo ; idempotent.
2. docs/ia/demo.md : déroulé minuté des 4 scénarios (planifier « 3 jours à Djerba, 300 DT, famille, culture + plage » ; modifier « je préfère moins marcher » ; justifier « pourquoi cette activité ? » ; transport « on part de Tunis en avion, 4 jours, 1200 DT à deux, avec une voiture » puis « finalement des transferts »), phrases exactes à taper, résultat attendu, plan B (Ollama hors-ligne, vidéo).
3. Diagrammes (Mermaid ou PlantUML) : architecture avec le service ai et les flux transport, séquence d'un message de l'agent, modèle de données voyara_ai ; exporte-les pour LaTeX.
4. Rédige un chapitre « Moteur IA » dans docs/rapport.tex à partir des rapports d'étape et de docs/ia/resultats/synthese.md : problématique, état de l'art (Tripadvisor, Booking, GetYourGuide, Google Travel, Kayak), architecture, méthodes par module, résultats chiffrés, comparaison des LLM, limites, perspectives (application mobile, interaction vocale, apprentissage des préférences). N'invente aucun chiffre : cite uniquement les résultats présents dans docs/ia/resultats/.
5. Vérifie que le projet démarre de zéro avec scripts/dev.ps1 et avec Docker (port 8080) et que la démo fonctionne dans les deux cas.

RAPPORT OBLIGATOIRE — à la fin, crée docs/ia/rapports/etape-13-demo-soutenance.md avec :
  1. Objectif de l'étape.
  2. Ce que tu as fait : chaque fichier créé/modifié, avec son rôle en une ligne.
  3. Choix techniques : pour CHAQUE décision, l'option retenue, les alternatives envisagées et pourquoi tu les as écartées.
  4. Vérification : commandes PowerShell exécutées et sortie réelle (extraits). N'écris jamais qu'un test passe si tu ne l'as pas lancé ; si quelque chose échoue, dis-le.
  5. Métriques obtenues (si applicable), avec le jeu de données utilisé.
  6. Limites, dette technique, risques.
  7. Ce que je dois vérifier moi-même (pas à pas).
  8. Étape suivante recommandée.
Mets aussi à jour docs/ia/journal.md (une ligne : date, étape, résumé) et coche la tâche dans docs/plan.md.
Termine en affichant le résumé du rapport dans le chat et propose un message de commit.

Annexe A — Glossaire IA

Terme	Définition
LLM	Grand modèle de langage (Claude, Qwen…) ; ici utilisé pour comprendre et expliquer, pas pour calculer.
Sortie structurée / tool use	Le LLM remplit un schéma JSON ou appelle une fonction au lieu d'écrire du texte libre.
Embedding	Vecteur numérique représentant le sens d'un texte ; deux textes proches ont des vecteurs proches.
pgvector	Extension PostgreSQL pour stocker des vecteurs et chercher les plus proches (index HNSW).
BM25	Score de recherche lexicale classique, fort sur les noms propres et mots exacts.
RRF	Reciprocal Rank Fusion : fusion de plusieurs classements par la somme de 1/(k + rang).
Recherche structurée	Recherche par filtres exacts (origine, destination, date, places) sur la source de vérité ; utilisée pour les vols, voitures et transferts.
RAG	Retrieval-Augmented Generation : le LLM répond à partir de documents récupérés dans nos données.
MMR	Maximal Marginal Relevance : re-classement qui équilibre pertinence et diversité.
Démarrage à froid	Absence d'historique pour un nouvel utilisateur ou une nouvelle activité.
CP-SAT	Solveur de programmation par contraintes d'OR-Tools (Google) pour les problèmes combinatoires.
Agent	Boucle où le LLM choisit des outils, observe le résultat et décide de l'étape suivante.
LLM-as-judge	Évaluation d'une sortie par un LLM selon une grille ; à recouper avec des humains.
Recall@k / nDCG	Métriques de recherche : part des documents pertinents trouvés dans le top k / qualité de l'ordre.
F1	Moyenne harmonique de la précision et du rappel, utilisée pour l'extraction de champs.

Annexe B — Checklist avant soutenance

☐ Les 4 scénarios (planifier, modifier, justifier, transport) fonctionnent, en ligne et hors-ligne.
☐ docs/ia/resultats/synthese.md régénéré avec les données finales.
☐ Comparaison Claude vs Ollama chiffrée (ou limite documentée si une seule source).
☐ Un rapport par étape dans docs/ia/rapports/ (dont 2-bis), journal à jour.
☐ Architecture et ADR à jour avec le périmètre transport.

☐ Démo lancée de zéro avec scripts/dev.ps1 et avec Docker (port 8080).
☐ Aucune clé API dans le dépôt (vérifier l'historique git).
☐ L'agent ne demande jamais passeport, permis ou carte bancaire.
☐ Interface vérifiée en fr, en et ar (RTL), mobile et desktop.
☐ Limites connues documentées dans docs/architecture.md.